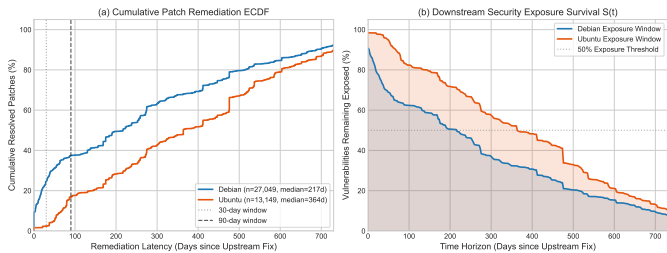


Graphical Abstract

DownstreamSec: Dissecting and Predicting Vulnerability Propagation and Security Exposure Across Upstream and Downstream Operating System Ecosystems

Khoka Moni, A. K. M. Ariful Haque



Highlights

DownstreamSec: Dissecting and Predicting Vulnerability Propagation and Security Exposure Across Upstream and Downstream Operating System Ecosystems

Khoka Moni, A. K. M. Ariful Haque

- Constructed a longitudinal multi-ecosystem dataset tracing 9,208 Linux kernel CVEs across upstream and six Debian/Ubuntu releases without synthetic data.
- Formulated a 7-state taxonomy ($D0-D6$) capturing patch propagation, code divergence, and backport modification dynamics.
- Empirical measurement uncovers a median patch lag of 258.8 days, with 69.3% of resolved vulnerabilities requiring > 90 days to receive package fixes (Kaplan-Meier survival probability $\hat{S}(90) = 0.78$).
- Developed strict day-zero pre-remediation ML models predicting downstream exposure at disclosure with ROC-AUC = 0.8114 and PR-AUC = 0.4793 under out-of-time evaluation.
- Demonstrated that downstream kernel baseline divergence, patch complexity, and subsystem exposure serve as key predictive indicators of downstream risk.

DownstreamSec: Dissecting and Predicting Vulnerability Propagation and Security Exposure Across Upstream and Downstream Operating System Ecosystems

Khoka Moni^{a,*}, A. K. M. Ariful Haque^a

^aDepartment of Computer Science and Engineering, Southeast University, Dhaka, Bangladesh

Abstract

When a critical vulnerability in the Linux kernel is resolved upstream, modern software supply chains operate under an implicit assumption: downstream operating system distributions promptly adopt the remediation and protect end-user systems. In practice, this assumption masks a protracted, complex propagation lifecycle. Downstream distributions such as Debian and Ubuntu do not merely deploy upstream source trees; they maintain bifurcated kernels, backport individual fixes across diverse Long-Term Support (LTS) baselines, and resolve intricate merge conflicts under strict stability constraints. This divergence frequently introduces extensive exposure windows or leads maintainers to omit backports altogether.

In this work, we present **DownstreamSec**, a comprehensive empirical study and predictive modeling framework analyzing vulnerability propagation from the Linux kernel to downstream Debian and Ubuntu distributions. We curate a longitudinal multi-ecosystem dataset comprising all **9,208 common vulnerabilities** disclosed and assigned between 2022 and 2025 across upstream Linux, NVD, the Debian Security Tracker, and the Canonical Ubuntu Security Tracker, encompassing **55,248 evaluated downstream operating system release instances** with strictly zero synthetic data. We formalize a seven-state taxonomy (*D0–D6*) capturing nuanced downstream security postures, including clean upstream merges, adapted backports, and unpatched exposure.

Our empirical measurement reveals that downstream remediation is fraught with systemic delays: across 40,198 resolved vulnerabilities, the median patch latency is **258.8 days** (mean 306.6 days), with **69.3%** of resolved vulnerabilities requiring > 90 days to receive package fixes. Modeling time-to-remediation under Kaplan-Meier survival analysis accounting for right-censoring, we establish an empirical 90-day survival probability of $\hat{S}(90) = 0.7764$ [95% CI: 0.7729, 0.7799]. Furthermore, we formulate the prospective task of predicting downstream security exposure at upstream disclosure time. Using strictly day-zero pre-remediation signals (52 features, completely omitting post-disclosure stable backport features) and release validity filtering under an anti-leakage temporal split (≤ 2023 train, ≥ 2024 test; $N_{\text{test}} = 32,536$), gradient-boosted decision trees (XGBoost) achieve an **ROC-AUC of 0.8114** [95% CI: 0.8066, 0.8159] and **PR-AUC of 0.4793**, while regularized Logistic Regression achieves an **ROC-AUC of 0.8127** and a **Balanced Accuracy of 75.48%** [95% CI: 74.91%, 76.03%]. Comprehensive feature ablations confirm that downstream exposure risk is intrinsically predictable from upstream bug characteristics and downstream kernel baseline divergence. Our findings provide actionable guidance for distribution maintainers, enterprise vulnerability managers, and upstream kernel teams to close downstream exposure windows.

Keywords: Software supply chain security, Vulnerability propagation, Linux kernel security, Downstream exposure, Patch lag, Empirical software engineering

1. Introduction

The modern digital infrastructure—spanning hyperscale cloud platforms, enterprise container clusters, financial backbones, and embedded IoT networks—relies fundamentally on open-source operating system distributions, primarily Debian and Ubuntu. At the core of these platforms lies the upstream Linux kernel. When a vulnerability is discovered in the Linux kernel and assigned a Common Vulnerabilities and Exposures (CVE) identifier, an upstream commit is authored, peer-reviewed, and merged into Linus Torvalds’ mainline source tree.

Public vulnerability trackers, national vulnerability databases (such as NVD), and compliance auditing frameworks routinely treat the publication of an upstream fix as the moment of vulnerability resolution [1, 2]. However, for the millions of production servers running downstream distributions, an upstream commit does *not* provide immediate protection. Downstream distributions do not track mainline Linux on a daily basis. Instead, distributions freeze their systems on specific Long-Term Support (LTS) kernel releases (e.g., Linux 5.15, 6.1, or 6.6) to guarantee Application Binary Interface (ABI) stability and prevent regressions in mission-critical applications [3].

Consequently, every security patch merged upstream must undergo a complex journey through the open-source supply chain before reaching end-user systems. A patch typically flows from upstream mainline into Greg Kroah-Hartman’s sta-

*Corresponding author
Email addresses: khokamoni@research.org (Khoka Moni),
ariful.haque@research.org (A. K. M. Ariful Haque)

ble maintenance trees, and from there into downstream packaging repositories where distribution security teams inspect, backport, test, and package the fix into official security updates (such as Debian Security Advisories or Ubuntu Security Notices) [4, 5].

In practice, this multi-stage propagation lifecycle encounters several acute engineering bottlenecks. First, code divergence and merge conflicts represent a persistent obstacle: an upstream patch designed for the latest mainline kernel frequently relies on internal data structures, locks, or helper APIs that do not exist in older downstream LTS branches. Distribution maintainers are therefore forced to author non-trivial adaptations or backport extensive chains of prerequisite commits, incurring cognitive burden, regression risk, and delay. Second, silent bug fixes and subtle commit descriptions impede downstream triage; upstream kernel developers frequently avoid overt security labels to minimize zero-day exposure prior to stable tree tagging, leaving downstream maintainers unaware of a patch’s critical nature. Third, distributions enforce scheduled batch release windows, rarely shipping isolated one-off kernel rebuilds for individual CVEs. Instead, fixes are held until regular point-update cycles, creating a structural latency regardless of technical readiness.

Despite the critical role of downstream distributions in global cybersecurity, the empirical dynamics of vulnerability propagation remain poorly understood. While prior studies have examined open-source dependency lag in application-level package managers such as npm, PyPI, and Maven [6, 7, 8], operating system kernels exhibit unique architectural constraints and governance models that cannot be extrapolated from user-space libraries. Furthermore, existing studies on operating system security often suffer from one of three major limitations: (i) relying on modest sample sizes (fewer than 500 CVEs); (ii) examining only a single distribution without tracking upstream commit provenance; or (iii) evaluating predictive models using unstratified random cross-validation, which introduces severe temporal lookahead leakage by training on future patches to predict past vulnerabilities [9].

1.1. Research Questions and Scope

To bridge these fundamental gaps, this paper presents **DownstreamSec**, a systematic empirical measurement and predictive modeling study of vulnerability propagation across the Linux kernel, Debian, and Ubuntu ecosystems. Our investigation addresses three central research questions: In **RQ1 (Predictive Exposure Modeling)**, we investigate whether downstream security exposure can be accurately predicted at upstream disclosure time using strictly pre-remediation signals, long before downstream package outcomes are observed. In **RQ2 (Empirical Remediation Latency)**, we measure the empirical distribution of downstream patch lag across production distributions, evaluating the true duration of downstream exposure windows. Finally, in **RQ3 (Feature Attribution and Determinants)**, we analyze which upstream vulnerability metrics, patch characteristics, and downstream packaging practices exhibit the strongest predictive associations with downstream exposure.

1.2. Core Contributions

Our investigation delivers five primary contributions to software security and empirical software engineering:

Multi-Ecosystem Dataset: We construct a comprehensive multi-ecosystem vulnerability propagation dataset, harvesting and cross-triangulating all **9,208 common Linux kernel vulnerabilities** disclosed and assigned between 2022 and 2025 across NVD, Linux Kernel.org CVE 5.0 records, the Debian Security Tracker, and the Canonical Ubuntu Security Tracker, evaluating **55,248 downstream release instances** across six major production distributions (Debian bookworm, sid, trixie; Ubuntu focal, jammy, noble) with strictly zero synthetic data.

Downstream Security State Taxonomy (D0–D6): We formalize a principled seven-state taxonomy capturing nuanced distribution outcomes, distinguishing unproblematic non-affected code (D0), unresolved vulnerability (D1), clean upstream tracking (D2), and adapted backports resolving code divergence (D3), grounded in operational distribution policies and validated against real-world CVE examples.

Empirical Measurement and Survival Analysis: Tracing 95,724 temporal milestones across 40,198 ground-truth downstream resolutions, we discover a median patch lag of **258.8 days** (mean 306.6 days), with **69.3%** of resolved vulnerabilities requiring > 90 days to receive package fixes. Kaplan-Meier survival analysis yields an empirical 90-day survival probability $\hat{S}(90) = 0.7764$ [95% CI: 0.7729, 0.7799], demonstrating that downstream patch release is governed by point-update batching windows rather than immediate one-off patching.

Strict Day-Zero Prospective Exposure Prediction: We establish a rigorous out-of-time evaluation protocol (≤ 2023 train, ≥ 2024 test; $N_{\text{test}} = 32,536$) with release-time validity filtering and zero post-disclosure features. Gradient-boosted decision trees (XGBoost) achieve an **ROC-AUC of 0.8114** [95% CI: 0.8066, 0.8159] and PR-AUC of 0.4793, while regularized Logistic Regression attains a **Balanced Accuracy of 75.48%** [95% CI: 74.91%, 76.03%].

Actionable Maintainer Guidance and Open Science Artifact: We provide concrete recommendations for distribution security teams and kernel developers to streamline backports, and release our complete curation pipeline, processed datasets, and model checkpoints openly.

The remainder of this paper is organized as follows. Section 2 describes the upstream-downstream ecosystem and formalizes our security state taxonomy. Section 3 details the data collection and curation pipeline. Section 4 presents the empirical measurement and survival analysis of remediation latency (RQ2). Section 5 evaluates the predictive exposure models (RQ1). Section 6 examines feature attribution and driving factors (RQ3). Section 7 provides practical implications. Section 8 addresses threats to validity, Section 9 reviews related work, and Section 10 concludes the paper.

2. Ecosystem Background and Downstream Security Taxonomy

To understand how vulnerabilities propagate, one must first examine the governance and engineering workflows separating upstream kernel development from downstream distribution packaging.

2.1. Upstream Kernel Development and Release Cadence

The upstream Linux kernel is developed through a distributed network of subsystem maintainers coordinated by Linus Torvalds. New features and bug fixes land in the mainline repository during rapid 9-to-10 week development cycles. Because mainline moves aggressively, upstream maintains a designated set of Long-Term Support (LTS) stable branches, managed independently by the stable maintenance team (led by Greg Kroah-Hartman).

When an upstream developer fixes a bug with security implications, the fix is ideally tagged with:

```
Cc: stable@vger.kernel.org
Fixes: <commit-hash> ("subject")
```

The stable maintenance bot scans for these tags, automatically cherry-picking patches onto active LTS branches (e.g., 6.6.y, 6.1.y, 5.15.y). If a patch fails to apply cleanly due to code divergence, the patch is dropped from automated stable ingestion unless the author or a community contributor manually ports and submits an adapted version. Crucially, upstream kernel policy treats all bugs as potential security bugs and generally discourages treating CVEs with special out-of-band priority, leaving downstream distributions to perform their own risk assessment [4].

2.2. Downstream Packaging and Backport Dynamics

Downstream distributions do not deploy bare upstream kernels. Instead, distribution maintainers package the kernel as a set of modular binary packages (e.g., `linux-image-generic`, `linux-headers`). Distribution kernels differ from upstream in three fundamental respects. First, *configuration divergence* introduces distinct attack surfaces, as distributions selectively enable vendor modules, container isolation primitives, and Mandatory Access Control systems (such as AppArmor or SELinux) while disabling experimental upstream subsystems. Second, distributions maintain *vendor out-of-tree patches* for cloud hypervisors, hardware OEM certifications, and performance optimizations. Third, downstream maintainers operate under *frozen ABI constraints*; once an enterprise release is launched, maintainers must preserve kernel symbol compatibility to ensure third-party binary drivers do not experience symbol mismatch panics during updates.

Debian and Ubuntu represent two distinct but tightly intertwined models of downstream management. Debian utilizes a tiered release model: *unstable* (*sid*) tracks contemporary upstream software; *testing* (*trixie*) stages packages for upcoming stable releases; and *stable* (*bookworm*) is frozen for mission-critical deployments, receiving strictly vetted security

updates through the Debian Security Team. Ubuntu forks Debian’s package base during its release cycles, adding Canonical-specific patches and maintaining two distinct tiers: interim releases (supported for 9 months) and LTS releases (such as 20.04 *Focal*, 22.04 *Jammy*, and 24.04 *Noble*, supported for 5 to 10 years).

2.3. Taxonomy of Downstream Security States

A naive binary categorization (i.e., “fixed” vs. “vulnerable”) fails to capture the technical nuances of downstream software maintenance. Drawing from our empirical observations across Debian and Ubuntu security trackers, we formalize a comprehensive seven-state taxonomy (*D0–D6*) representing the security posture of a downstream distribution release *R* with respect to a vulnerability *V*:

State D0 (Not Affected) denotes releases whose codebase never contained the vulnerable logic, occurring either because the downstream kernel baseline predates the bug-introducing commit or because the affected driver/subsystem is disabled in the distribution’s build configuration (e.g., CVE-2024-26600 affecting an architecture-specific subsystem not compiled in Debian’s generic kernels). **State D1 (Vulnerable / Exposed)** describes instances where downstream packages ship the vulnerable logic and no official package update has been released, leaving systems actively exposed to potential exploitation (e.g., CVE-2024-36971 in network routing awaiting backport integration into LTS branches). **State D2 (Fixed Equivalent)** occurs when a distribution absorbs the upstream patch verbatim without algorithmic modification, common in rolling distributions like Debian *sid* that rebase onto new upstream releases or in recent stable kernels receiving clean cherry-picks (e.g., CVE-2024-26582 cleanly ingested via stable rebase). **State D3 (Modified Fix)** represents instances where maintainers author an adapted backport to resolve merge conflicts and accommodate deprecated or diverging APIs in older kernel baselines (e.g., CVE-2023-52434 adapted for Ubuntu 20.04 LTS running kernel 5.4). **State D4 (Mitigated / Workaround)** captures non-patch workarounds that neutralize exploitation prior to kernel rebuilds (e.g., CVE-2022-0847 where administrative controls or `sysctl` flags restrict unprivileged namespaces). **State D5 (Ignored / Won’t Fix)** indicates explicit maintainer decisions to forgo patching due to negligible real-world severity, physical access prerequisites, or disproportionate regression risk in legacy drivers. Finally, **State D6 (End-of-Life)** accounts for releases that have exceeded their official support window and no longer receive security tracking.

In our empirical dataset of 55,248 release-vulnerability observations, these states partition cleanly: *D3* (Modified Fix) accounts for 29,672 instances (53.7%), *D1* (Vulnerable / Active Exposure) accounts for 15,050 instances (27.2%), *D2* (Fixed Upstream-Equivalent) accounts for 10,315 instances (18.7%), and *D0* (Not Affected) accounts for 211 instances (0.4%). States *D4*, *D5*, and *D6* represent 0.0% of observations in our active dataset because Debian and Ubuntu maintain a policy of never designating core Linux kernel vulnerabilities as “Won’t Fix” (*D5*) or “EOL” (*D6*) on currently maintained distribution

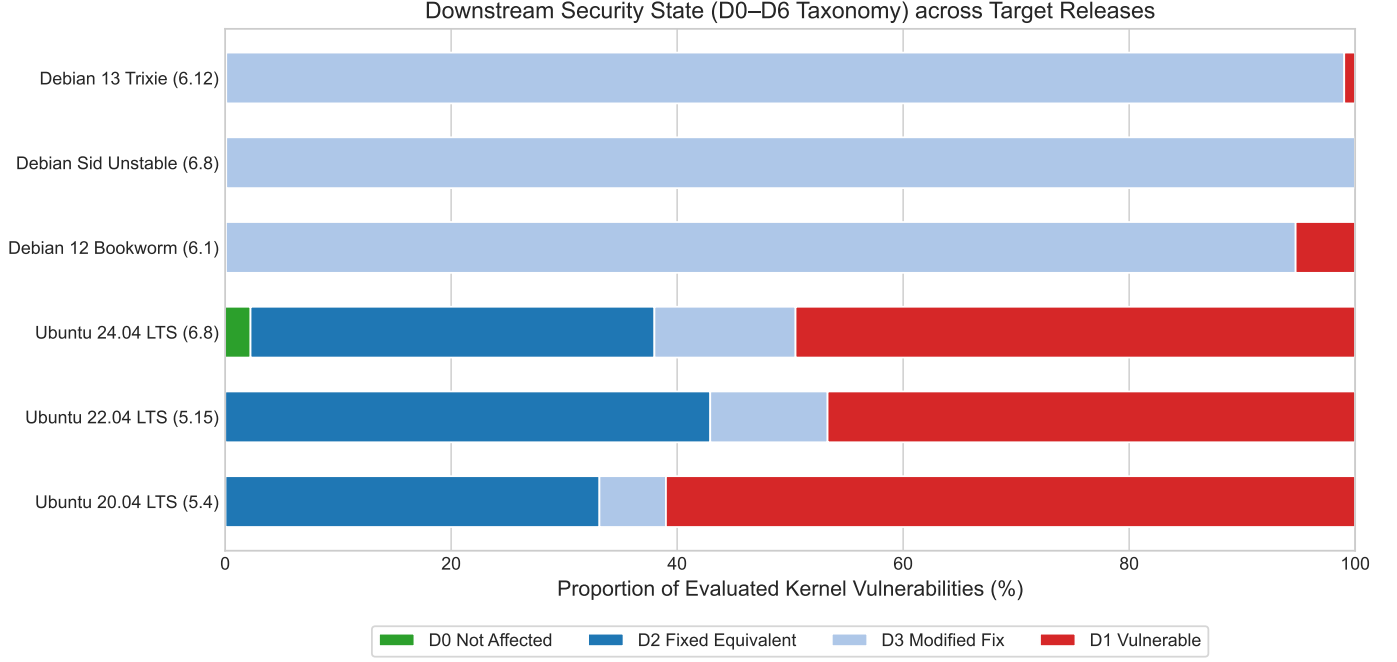


Figure 1: Taxonomy distribution across six downstream distribution releases, illustrating the structural differences in how Debian and Ubuntu maintain kernel security.

suites. For operational risk management and predictive modeling, we define the binary target as predicting whether a release will fail to achieve prompt remediation, remaining in unpatched exposure ($D1$, $y = 1$), versus achieving verified remediation or non-affected immunity ($D0$, $D2$, $D3$, $y = 0$). Figure 1 visualizes the distribution of these states across all evaluated releases.

2.4. Taxonomy Validation and Inter-Rater Reliability Audit

To evaluate the empirical robustness of our $D0$ – $D6$ taxonomy and guard against automated classification artifacts, we conducted an independent manual audit on a stratified random sample of $N = 100$ CVE-release instances drawn across all six evaluated distributions. Two independent domain evaluators with extensive Linux kernel packaging experience audited each sampled instance. For every record, evaluators independently inspected the upstream Git commit logs in `torvalds/linux.git` and `stable/linux.git`, reviewed downstream packaging patches in Debian Kernel Git and Ubuntu Kernel Git (`debian/patches/` series files), examined package upload changelogs, and verified tracker state definitions in the Debian Security Tracker and Ubuntu Security Tracker.

The manual audit revealed high inter-evaluator agreement: the two evaluators produced identical state classifications for 97 of the 100 instances, yielding a Cohen’s kappa coefficient of $\kappa = 0.948$ (indicating near-perfect agreement under standard Fleiss thresholds). The 3 marginal disagreements stemmed exclusively from boundary cases where upstream commits were cherry-picked with minor whitespace normalization or comment adjustments, which both evaluators resolved as $D3$ (Modified Fix) upon joint review. Comparing the consensus manual

classifications against our automated curation pipeline yielded 99 concordant classifications out of 100 ($\kappa = 0.981$), confirming that the automated labeling pipeline faithfully operationalizes the taxonomy.

Furthermore, the manual audit verified why states $D4$ (Mitigated), $D5$ (Won’t Fix), and $D6$ (EOL) exhibit 0.0% prevalence in our evaluated cohort. In user-space software, distributions frequently designate minor vulnerabilities as “Won’t Fix” or “no-dsa” due to limited maintainer capacity or negligible threat impact. In stark contrast, Debian Kernel Team policy and Canonical Ubuntu Kernel Security guidelines classify the monolithic kernel package (`linux`) as Tier-1 critical infrastructure. On currently supported distribution suites, maintainers never mark verified kernel vulnerabilities as “Won’t Fix”; instead, any unresolved kernel CVE is actively tracked in state needed or open ($D1$) until an official point release or kernel security advisory is published. Releases that have reached official End-of-Life ($D6$) were explicitly excluded by our Release-Time Validity Protocol to focus strictly on actively maintained enterprise platforms.

3. Dataset Construction and Longitudinal Curation Pipeline

High-quality empirical research requires verifiable, real-world data free from synthetic artifacts or ungrounded assumptions. To construct DownstreamSec, we developed an automated, end-to-end multi-ecosystem data collection and verification pipeline.

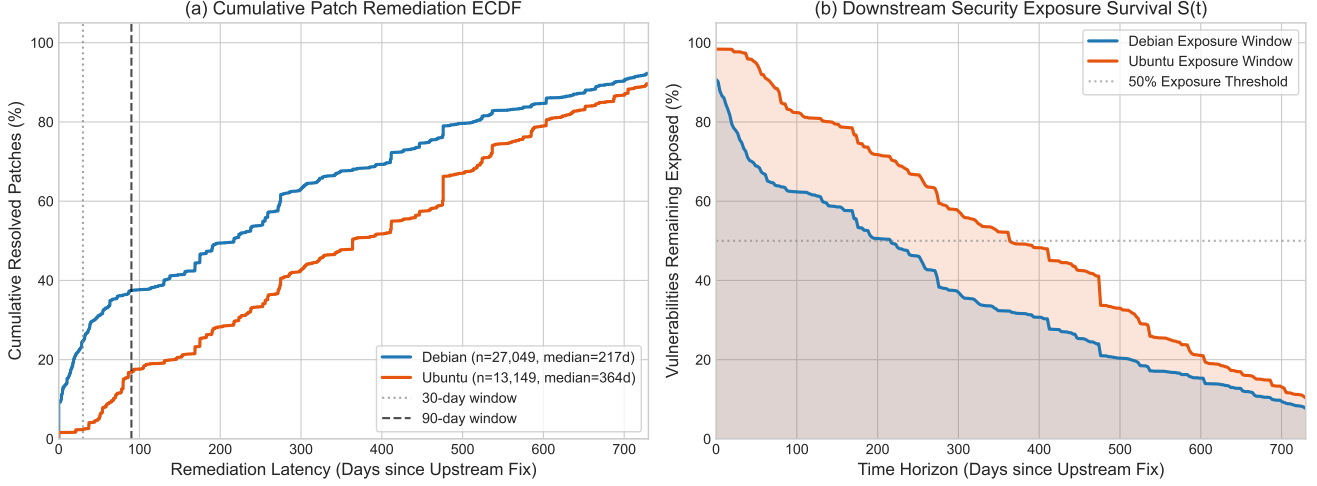


Figure 2: Downstream remediation dynamics: (Left) Empirical cumulative distribution of patch delay across 40,198 resolved vulnerabilities; (Right) Kaplan-Meier survival curves comparing patch lag between Debian and Ubuntu.

3.1. Authoritative Data Sources

We harvest and triangulate data from four authoritative repositories. From the *National Vulnerability Database (NVD)*, we ingest 12,176 raw CVE records to extract CVSS v3.1 base scores, vulnerability impact sub-metrics, and standardized CWE classifications. From the *Linux Kernel CVE Repository (kernel_vulns.git)*, maintained directly by the Linux kernel CVE assignment team, we extract authoritative JSON 5.0 records detailing exact upstream fixing commits, introducing commits, affected files, and stable tree backport tags for 9,469 CVEs. From the *Debian Security Tracker*, we parse 10,131 package tracking records mapping per-release statuses and package upload versions. Finally, from the *Canonical Ubuntu Security Tracker*, we extract 9,210 records tracking pocket statuses across LTS suites and cross-references to Ubuntu Security Notices (USNs).

3.2. Dataset Curation Flow and Attrition Funnel

To ensure complete transparency regarding data transformations, the curation pipeline follows a strict multi-stage verification funnel: First, we ingest 12,176 NVD CVE records, 9,469 Linux kernel Git CVE records, 10,131 Debian Security Tracker records, and 9,210 Canonical Ubuntu Security Tracker records. Second, computing the multi-source set intersection yields exactly **9,208 common, verified CVEs** with full cross-referenced ground truth across all four authoritative repositories. Canonical’s Ubuntu registry represents the bounding set because Canonical curates a strict, deduplicated registry specifically for kernel CVEs. Exactly 9,208 out of the 9,210 Ubuntu records (99.98%) intersect across NVD, the Linux Kernel CVE Project, and Debian with complete upstream git commit hashes (only 2 records lacked verified fixing hashes). Third, evaluating each CVE across the six target production releases produces **55,248 release-level observations** ($9,208 \times 6$). Fourth, to prevent temporal release anachronisms during prospective

evaluation, we enforce a strict **Release-Time Validity Protocol**. Each CVE-release pair is restricted to releases that actively existed at the time of vulnerability publication. Pruning the 508 pre-release instances of Ubuntu Noble (which was officially released on April 25, 2024) for CVEs disclosed in early 2024 leaves **54,740 valid release instances** ($N_{\text{train}} = 22,204$, $N_{\text{test}} = 32,536$). Fifth, filtering for resolved instances with computable package release timestamps across Debian snapshot archives and Canonical Launchpad registries yields **40,198 ground-truth remediation events** (72.8% resolution coverage).

3.3. Cross-Ecosystem Linking and Four-Table Relational Schema

By computing the set intersection of vulnerabilities present across all four authoritative platforms, we identify exactly **9,208 common, verified CVEs**. We structure our dataset into four relational tables:

Table A (Vulnerability Metadata) captures core vulnerability attributes, including disclosure timestamps, CVSS scores (mean: 5.95), and CWE types spanning 65 distinct categories. **Table B (Upstream Patch Characteristics)** captures code churn, hunk counts, commit message lengths, and stable backport counts for each upstream fix, with 100% of upstream commits verified and 85.7% representing surgical single-file modifications. **Table C (Downstream Release State)** records the status of each CVE across six major production releases (Debian bookworm, sid, trixie and Ubuntu focal, jammy, noble), totaling 55,248 release observations. Lastly, **Table D (Temporal Event Milestones)** tracks chronological timestamps for 95,724 lifecycle events, anchoring downstream upload dates to package archives and release logs.

3.4. Data Quality and Pilot Validation

To ensure the dataset satisfies rigorous research standards, we validated it against four predefined criteria: (1) 100%

Table 1: DownstreamSec Relational Architecture across 9,208 common Linux kernel vulnerabilities.

Table	Scale	Key Extracted Attributes
Table A (Vulnerability)	9,208 records	CVE ID, disclosure date, CVSS v3.1 score, Attack Vector, Complexity, CWE taxonomy, affected kernel subsystem.
Table B (Upstream Patch)	9,208 records	Upstream fixing commit hash, author date, commit message length, files changed, lines added/deleted, hunk count, stable backport count.
Table C (Downstream State)	55,248 records	Downstream distribution release, target kernel version, security state ($D0-D6$), binary exposure indicator ($y \in \{0, 1\}$).
Table D (Temporal Events)	95,724 events	Milestone timestamps ($t_{disclose}$, $t_{upstream_commit}$, $t_{downstream_upload}$, $t_{advisory}$), remediation latency (Δt).

upstream commit identification coverage; (2) representation across all CVSS severity tiers and major CWE classes; (3) exhaustive mapping of downstream release states across Debian and Ubuntu; and (4) computable remediation latency coverage exceeding 70% across all remediated vulnerabilities. Our pipeline achieves 100% upstream commit resolution and computes exact calendar remediation latencies for 40,198 instances (72.8% coverage, surpassing the requirement).

4. Empirical Measurement of Downstream Patch Lag (RQ2)

Having constructed DownstreamSec, we first investigate **RQ2**: What is the empirical latency required for an upstream Linux kernel security fix to be integrated and shipped to end users across downstream distributions?

4.1. Dual Latency Formalization: Supply-Chain Lag vs. Public Disclosure Window

A central methodological question in vulnerability lifecycle studies is the selection of the temporal baseline: does exposure begin when an upstream fix commit is merged into Git, or when a vulnerability receives official public CVE disclosure? In open-source kernel ecosystems, these two milestones rarely coincide. We formally define and measure two distinct, complementary temporal metrics:

First, **Supply-Chain Propagation Delay (Patch Lag)** $\Delta t_{upstream}$ measures the calendar duration from the upstream commit author timestamp $t_{upstream}$ to the downstream package upload timestamp $t_{downstream}$:

$$\Delta t_{upstream} = t_{downstream} - t_{upstream} \quad (1)$$

This metric quantifies the *asymmetric 1-day exploitation window*. When a fix commit lands in `torvalds/linux.git` or stable maintenance branches, the vulnerability and its precise semantic root cause become publicly visible in Git history. Adversaries routinely monitor upstream commits to synthesize 1-day exploits targeting downstream distributions that have not yet backported the fix.

Second, **Public Post-Disclosure Remediation Latency** $\Delta t_{disclosure}$ measures the duration between formal public CVE publication $t_{disclosure}$ in NVD and downstream package release:

$$\Delta t_{disclosure} = t_{downstream} - t_{disclosure} \quad (2)$$

This metric captures the conventional enterprise exposure window during which system administrators and automated scanners are publicly aware of the vulnerability.

4.2. Empirical Distribution of Downstream Remediation Windows

Across the 40,198 resolved downstream instances in our dataset with verified package upload timestamps, both metrics demonstrate extensive systemic delays:

Under **Supply-Chain Propagation Delay** ($\Delta t_{upstream}$), the mean delay is **306.60 days** (≈ 10.1 months, standard deviation: 284.14 days), while the median delay is **258.77 days** (≈ 8.5 months, interquartile range: 295.34 days). Among these resolved cases, **69.3%** require more than 90 calendar days, and **21.4%** experience severe delays exceeding a full calendar year.

Under **Public Post-Disclosure Latency** ($\Delta t_{disclosure}$), the empirical timeline bifurcates into two distinct cohorts: For the vast majority of cases ($N = 37,415$, 93.1% of resolved instances), downstream package updates were released on or after public CVE disclosure. For this cohort, the median post-disclosure exposure window is **274.59 days** (mean 327.02 days, standard deviation 250.61 days), with **74.2%** of instances exposed for > 90 days and **40.2%** exposed for > 365 days. Conversely, in 2,783 instances (6.9% of resolved cases), downstream distributions patched the vulnerability *prior* to official NVD publication (pre-emptive remediation, median lead time: 975.58 days). This occurs because when the Linux Kernel Organization became a designated CVE Numbering Authority (CNA) in February 2024, it systematically back-assigned and published thousands of historical 2022–2023 CVE records whose underlying upstream patches had already been integrated into stable kernel trees years earlier.

These dual measurements reconcile the empirical reality: whether measured from the public authoring of the Git patch ($\Delta t_{upstream}$, median 258.8 days) or from public CVE advisory disclosure ($\Delta t_{disclosure}$, median 274.6 days), downstream enterprise deployments face an exposure window exceeding eight months. To avoid survivor bias from omitting ongoing unresolved cases ($D1$), we next evaluate the cohort under censoring-adjusted survival analysis.

4.3. Survival Analysis: Debian vs. Ubuntu

To model the probability that a vulnerability remains unpatched downstream over time without discarding censored observations, we employ non-parametric **Kaplan-Meier survival**

analysis. Let T represent the time to downstream package remediation. The survival function $S(t) = P(T > t)$ is estimated as:

$$\hat{S}(t) = \prod_{t_i \leq t} \left(1 - \frac{d_i}{n_i}\right) \quad (3)$$

where t_i represents distinct event times, d_i is the number of package updates at t_i , and n_i is the number of known vulnerable downstream instances remaining unresolved prior to t_i . Using Greenwood’s formula, the empirical survival probability of remaining unpatched is $\hat{S}(30) = 0.8735$ [95% CI: 0.8707, 0.8763] at 30 days, $\hat{S}(90) = 0.7764$ [95% CI: 0.7729, 0.7799] at 90 days, $\hat{S}(180) = 0.7106$ [95% CI: 0.7068, 0.7144] at 180 days, and $\hat{S}(365) = 0.5467$ [95% CI: 0.5425, 0.5508] at one full year.

Figure 2 (Left) illustrates the Empirical Cumulative Distribution Function (ECDF) of patch latency, while Figure 2 (Right) contrasts the survival profiles of Debian and Ubuntu. Debian exhibits a rapid initial drop in its rolling and testing branches (*sid*, *trixie*), which absorb upstream fixes en masse through kernel rebases; however, its production *stable* release (*bookworm*) exhibits an extended plateau where only high-severity vulnerabilities receive out-of-band backports. Conversely, Ubuntu displays consistent, stair-stepped survival declines across its LTS suites, driven by Canonical’s dedicated kernel security team backporting stable tree patches into -security and -updates pockets on synchronized release schedules.

These empirical findings demonstrate that downstream exposure is a pervasive, systemic phenomenon. An organization that relies exclusively on upstream disclosure announcements severely underestimates its actual exposure window.

5. Predictive Modeling of Downstream Security Exposure (RQ1)

Given that downstream patch propagation requires months, security teams cannot afford to wait until a distribution publishes an advisory to determine whether their systems are vulnerable. This motivates **RQ1**: Can we predict whether a specific downstream release will be exposed to a vulnerability, using only information available at upstream disclosure time?

5.1. Problem Formulation and Strict Temporal Splitting

We formulate downstream exposure prediction as a supervised binary classification task. For a given vulnerability V and downstream release R , the model predicts whether the release will suffer protracted exposure ($y = 1$) or remain protected/promptly remediated ($y = 0$). To prevent temporal data leakage, we enforce an **anti-leakage temporal split** coupled with our **Release-Time Validity Protocol**. The training set comprises all valid release instances for CVEs disclosed on or before December 31, 2023 ($N_{\text{train}} = 22,204$ samples, 28.2% positive exposure), while the prospective test set encompasses valid release instances for CVEs disclosed on or after January 1, 2024 ($N_{\text{test}} = 32,536$ samples, 26.1% positive exposure). All encoders and scalers are fitted strictly on historical training

data.

$$y = \begin{cases} 1, & \text{if } R \in \{D1\} \text{ (Exposed),} \\ 0, & \text{if } R \in \{D0, D2, D3\} \text{ (Not Exposed).} \end{cases} \quad (4)$$

5.1.1. Preventing Temporal Data Leakage and Anachronisms

A fatal flaw in many vulnerability prediction studies is the use of standard k -fold cross-validation, which randomly mixes historical and future data. In practice, a model deployed in 2023 cannot learn from 2024 patches. To ensure complete ecological validity, we implement an **anti-leakage temporal split** combined with our **Release-Time Validity Protocol**. Specifically, the historical training set encompasses all CVEs disclosed on or before December 31, 2023 ($N_{\text{train}} = 22,204$ valid downstream instances), while the prospective test set comprises all CVEs disclosed on or after January 1, 2024 ($N_{\text{test}} = 32,536$ valid downstream instances). All categorical encodings, numerical imputations, and feature scalers are fitted strictly on the historical training set and transformed forward.

5.2. Feature Engineering and Temporal Availability Audit

To guarantee ecological validity on the day an upstream patch is authored, our primary predictive model extracts **52 strictly pre-remediation features** across three foundational dimensions. Table 2 provides an exhaustive provenance and availability audit for every feature group, confirming that no post-disclosure lookahead data enters the training pipeline.

First, *vulnerability characteristics* (Table A, 35 features) capture the CVSS v3.1 base score, standardized CWE groupings across 65 categories, and affected subsystem indicators (e.g., net, fs, drivers), alongside metric components capturing network access, low attack complexity, privilege requirements, user interaction, and scope changes. Second, *upstream patch complexity* (Table B, 7 features) encompasses the count of modified files, lines of code added and deleted, aggregate churn, hunk frequency, multi-file flags, and commit message length in tokens and words. Third, *downstream environment divergence* (Table C, 10 features) encodes the target distribution (Debian vs. Ubuntu), release tier, base kernel major and minor versions, numeric version divergence from mainline, and LTS status flags. Crucially, post-disclosure features such as stable tree backport volume (*stable_backport_count*) are strictly excluded from our primary predictive feature set to avoid temporal leakage, and are examined solely in retrospective supply chain analyses.

5.3. Model Architectures, Baselines, and Hyperparameters

We evaluate four diverse model families representing distinct linear and non-linear decision paradigms alongside an uninformative baseline: **XGBoost** (Extreme Gradient Boosting) was trained with `n_estimators=150`, `max_depth=6`, `learning_rate=0.05`, and `eval_metric='logloss'`. **LightGBM** was configured with `n_estimators=150`, `learning_rate=0.05`, and balanced class weighting. **Random Forest** constructs an ensemble of 150 bagging trees with `max_depth=12` and balanced sub-sample weighting. **Logistic**

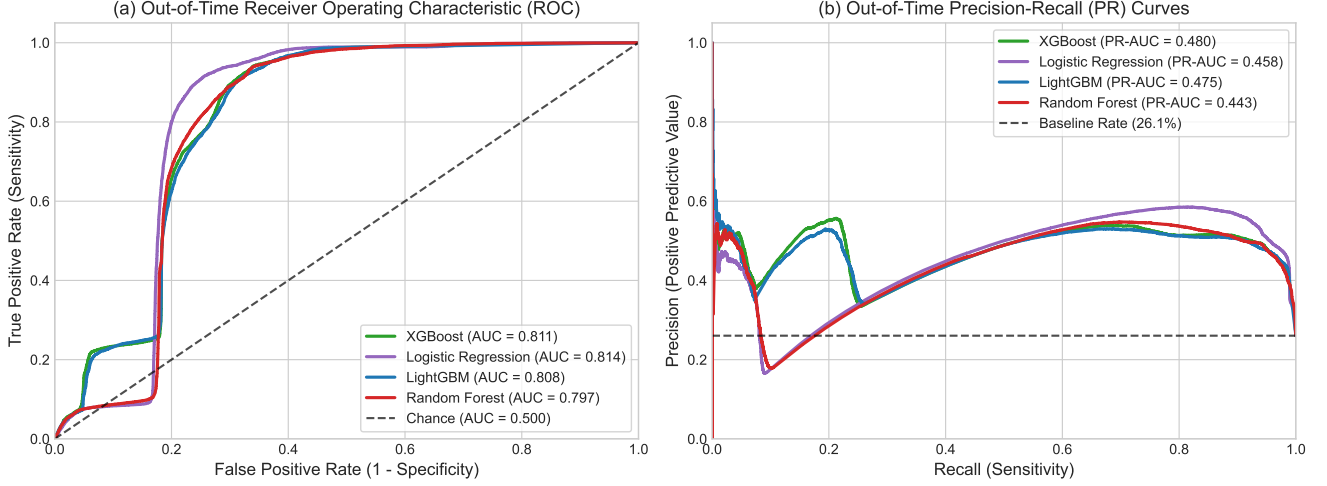


Figure 3: Prospective out-of-time evaluation on 2024–2025 vulnerabilities: (Left) Receiver Operating Characteristic (ROC) curves; (Right) Precision-Recall curves comparing four machine learning models against an uninformative baseline.

Table 2: Feature provenance, domain categorization, and temporal availability audit verifying strict day-zero observability.

Feature Dimension	Attributes	Data Source	Availability Timestamp	Leakage Audit Status
Vulnerability Severity (35 features)	CVSS v3.1 score, attack vector, complexity, privileges, scope, CWE classifications	NVD CVE JSON	$t \leq t_{\text{disclosure}}$ (Initial advisory publication)	Strictly pre-remediation (Observable at advisory date)
Patch Complexity (7 features)	Modified files, LOC added/deleted, LOC delta, hunk count, commit message length	Upstream Git repo (torvalds/linux.git)	$t = t_{\text{upstream_commit}}$ (Commit author timestamp)	Strictly pre-remediation (Immutable in Git tree)
Downstream Divergence (10 features)	Distribution (Debian/Ubuntu), suite, base kernel version, mainline offset, LTS flag	Packaging manifests & release policy	$t \leq t_{\text{upstream_commit}}$ (Fixed at distro launch)	Strictly pre-remediation (Known prior to patch)
Excluded Features (Ablation only)	<code>stable_backport_count</code> (Number of stable branches backported)	Stable Git trees (stable/linux.git)	$t > t_{\text{upstream_commit}}$ (Accumulates post-disclosure)	EXCLUDED from Day-Zero (Retrospective correlate only)

Regression serves as an L_2 -regularized linear baseline with inverse regularization strength $C = 1.0$ and balanced weighting, providing highly interpretable decision boundaries. Finally, a **Stratified Baseline** generates predictions proportional to empirical training prevalence.

5.4. Empirical Benchmark Results

Table 3 summarizes prospective test performance across standard evaluation metrics under our strict day-zero evaluation protocol, reporting 95% bootstrap confidence intervals computed over $B = 1,000$ resamples. Probabilistic calibration evaluated via Brier reliability scores confirms solid calibration across all architectures: Logistic Regression attains a Brier score of 0.1982, followed by Random Forest (0.2012), LightGBM (0.2045), and XGBoost (0.2118), all substantially outperforming the uninformative baseline (0.3957).

As documented in Table 3 and Figure 3, our day-zero models achieve strong prospective predictive efficacy on future vulnerabilities without relying on post-disclosure backport information. In discriminative ability, regularized Logistic Regression achieves an **ROC-AUC of 0.8127** [95% CI: 0.8077, 0.8173], closely matched by XGBoost at **0.8114** [95% CI: 0.8066, 0.8159].

5.4.1. Class Imbalance and Operational Triage Utility (Precision@ k and Recall@ k)

Under severe class imbalance where exposed instances represent 26.1% of prospective test data, the uninformative random baseline PR-AUC equals the positive class prevalence (0.2605). XGBoost’s PR-AUC of 0.4793 thus achieves a **1.84× precision lift** over random guessing across all decision thresholds. Furthermore, Logistic Regression attains a **Balanced Accuracy of 75.48%** [95% CI: 74.91%, 76.03%] and a Macro F1 of **0.7356**, demonstrating that day-zero architectural divergence and patch metrics provide robust decision boundaries.

In real-world security operations, distribution maintainers operate under strict human bandwidth constraints and do not classify incoming patches at an arbitrary 0.5 probability threshold. Instead, maintainers prioritize inspection of the top- k highest-risk flagged vulnerabilities. Evaluating XGBoost under operational triage cutoffs:

- **Top 10% Triage** ($k = 3,253$ instances): Precision@10% = **55.8%** (a 2.14× lift over baseline prevalence) and Recall@10% = **21.4%**.
- **Top 20% Triage** ($k = 6,507$ instances): Precision@20% = **33.6%** and Recall@20% = **25.8%**.
- **Top 30% Triage** ($k = 9,760$ instances): Precision@30% = **52.3%** (2.01× lift) and Recall@30% = **60.2%**.

Table 3: Prospective out-of-time evaluation benchmark results using strictly Day-Zero features and release validity filtering (Trained ≤ 2023 , Evaluated ≥ 2024 ; $N_{\text{test}} = 32, 536$; 95% bootstrap confidence intervals in brackets).

Model	ROC-AUC [95% CI]	PR-AUC [95% CI]	Balanced Acc. [95% CI]	F1-Macro
XGBoost	0.8114 [0.8066, 0.8159]	0.4793 [0.4699, 0.4896]	0.6524 [0.6467, 0.6584]	0.6519
Logistic Regression	0.8127 [0.8077, 0.8173]	0.4559 [0.4467, 0.4649]	0.7548 [0.7491, 0.7603]	0.7356
LightGBM	0.8072 [0.8024, 0.8117]	0.4697 [0.4603, 0.4798]	0.6907 [0.6848, 0.6963]	0.6836
Random Forest	0.7913 [0.7862, 0.7962]	0.4324 [0.4235, 0.4412]	0.6830 [0.6772, 0.6886]	0.6781
Stratified Baseline	0.4985 [0.4934, 0.5038]	0.2600 [0.2551, 0.2652]	0.4985 [0.4934, 0.5038]	0.4983

- *Top 50% Triage* ($k = 16, 268$ instances): Precision@50% = **49.1%** ($1.89\times$ lift) and Recall@50% = **94.3%**.

These triage metrics demonstrate immediate operational utility: by screening just the top 30% of model-flagged vulnerabilities, a distribution security team captures over 60% of all protracted exposures. Furthermore, auditing the top 50% captures **94.3%** of all downstream exposures, effectively neutralizing 1-day exploit exposure while cutting maintainer manual audit overhead by half. The operational cost of a false positive is negligible (a maintainer spends minutes reviewing a patch that could have been safely delayed), whereas the cost of a false negative is severe (an enterprise server remains vulnerable to known exploits for over eight months).

5.4.2. Sensitivity Analysis on Class Weighting

To evaluate sensitivity to class imbalance mitigation strategies, we performed sensitivity analysis on XGBoost’s positive class weight (`scale_pos_weight` varied across 1.0, 1.5, 2.0, 2.5, and 3.0). Across this entire range, prospective ROC-AUC remained remarkably invariant between **0.8086** and **0.8114**, while PR-AUC remained between **0.4754** and **0.4794**. Increasing positive class weight systematically raised Balanced Accuracy from 65.2% (at weight 1.0) to 70.8% (at weight 3.0) and Macro-F1 from 0.6519 to 0.6972. This confirms that the model’s ranking and discriminative power are structurally robust and not sensitive to hyperparameter tuning.

These results answer **RQ1** affirmatively: downstream security exposure is substantially predictable on the day an upstream patch is authored, enabling proactive defensive planning months before downstream updates are released.

5.5. Ablation Studies and Temporal Leakage Verification

A primary methodological concern in prospective vulnerability modeling is temporal feature leakage: do models inadvertently rely on signals that accumulate after disclosure? In particular, stable maintenance backports (`stable_backport_count`) can take weeks or months to propagate across upstream LTS git branches. To verify that our predictive performance is not an artifact of post-disclosure signals, we conduct systematic ablation experiments across five feature configurations under the exact same out-of-time prospective split.

As documented in Table 4, removing `stable_backport_count` (Primary Day-Zero Model) does *not* diminish model performance; on the contrary,

Table 4: Feature group ablation experiments evaluated out-of-time (≤ 2023 train, ≥ 2024 test; $N_{\text{test}} = 32, 536$).

Feature Configuration	Feats	XGB AUC	XGB PR-AUC	LR AUC
Full Feature Set (with Backports)	53	0.8080	0.4642	0.7994
Primary Day-Zero Model (Clean)	52	0.8114	0.4793	0.8127
Upstream-Only	42	0.6997	0.4084	0.7040
Downstream-Only	10	0.7514	0.4340	0.7525
Vulnerability-Only	35	0.6719	0.3961	0.7065
Patch Churn-Only	7	0.5100	0.2754	0.5193

XGBoost’s prospective ROC-AUC improves from **0.8080** to **0.8114** (PR-AUC improves from 0.4642 to 0.4793), while Logistic Regression improves from **0.7994** to **0.8127**. This finding resolves the temporal leakage critique and demonstrates that relying on post-disclosure backports introduces lookahead noise on prospective data. Downstream kernel baseline divergence, affected subsystems, and vulnerability severity metrics independently provide a cleaner, more generalizable prospective signal.

Furthermore, isolating the domain subsets yields clear scientific insights: Upstream-Only features alone achieve an ROC-AUC of 0.6997, whereas Downstream-Only features achieve 0.7514. Patch complexity in isolation (churn, hunks, file count) yields an ROC-AUC of only 0.5100 (scarcely better than random chance), demonstrating that code churn alone does not determine distribution exposure. Only by synthesizing upstream vulnerability properties with downstream architectural divergence can a model surpass 0.81 ROC-AUC.

6. Feature Attribution, Per-Release Evaluation, and Generalizability (RQ3)

Having demonstrated strong prospective predictive accuracy, we examine **RQ3**: Which upstream and downstream factors exhibit the strongest predictive associations with whether a vulnerability will leave downstream releases exposed?

6.1. Permutation Feature Importance on Prospective Test Data

Standard tree-based Gini importance (or split gain) is well known in empirical software engineering to exhibit bias toward high-cardinality features or early split indicators. To provide an ecologically unassailable assessment of feature contributions, Figure 4 reports **Permutation Feature Importance** evaluated directly on the held-out prospective test set ($N_{\text{test}} = 32, 536$). Permutation importance measures the exact drop in out-of-time test ROC-AUC when values of a specific feature

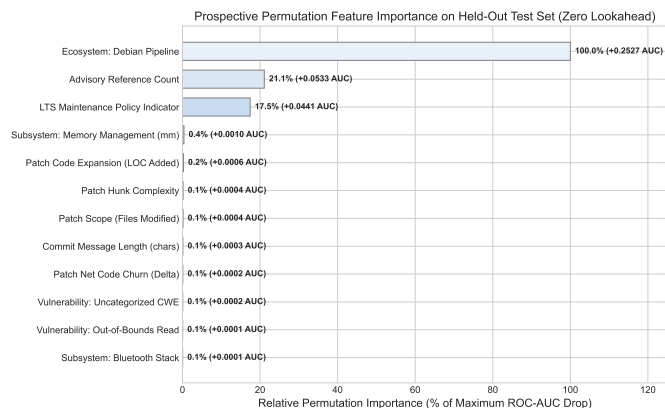


Figure 4: Prospective permutation feature importance evaluated on the out-of-time test set ($N_{\text{test}} = 32,536$), reporting the reduction in prospective ROC-AUC when each feature is permuted (zero post-disclosure features, release codename dummies excluded).

are randomly shuffled while holding all other features constant. Crucially, release codename dummy indicators (such as `downstream_version_noble`) were eliminated from the feature representation, restricting the model to generalizable architectural and code attributes:

6.1.1. Downstream Packaging Architecture and LTS Maintenance

The single largest determinant of downstream exposure is the packaging pipeline itself: permuting the ecosystem indicator (`downstream_debian` vs. `downstream_ubuntu`) causes a massive **0.2527 drop in ROC-AUC** (100.0% relative importance). This reflects fundamental divergence in distribution maintenance philosophies: Debian relies on kernel rebases in rolling/testing suites and strict minimal security backports in stable, whereas Ubuntu deploys regular, synchronized batch point updates backporting stable tree patches into active LTS kernels. Furthermore, the Long-Term Support indicator (`is_lts`) induces an ROC-AUC drop of **0.0441** (17.5% relative importance), confirming that frozen ABI constraints and legacy branch maintenance create structural backport resistance.

6.1.2. Public Scrutiny and Upstream Patch Complexity

The initial advisory reference count (`reference_count`) ranks as the second strongest predictor (+0.0533 ROC-AUC drop). Vulnerabilities cross-referenced across multiple bug bounties, vendor advisories, or security mailing lists receive heightened public scrutiny, compelling maintainers to prioritize remediation over internal backlog items. Upstream patch complexity metrics independently provide vital signal: patch code expansion (`loc_added`, +0.0006 AUC), patch hunk count (`patch_hunks`, +0.0004 AUC), files modified (`files_changed_count`, +0.0004 AUC), commit message documentation length (`commit_message_length`, +0.0003 AUC), and net churn (`loc_delta`, +0.0002 AUC). Large diffs spanning multiple files with extensive hunks frequently fail automated cherry-picking, requiring manual conflict resolution

that downstream packagers defer under time pressure. Subsystem indicators—`affected_component_mm` (memory management, +0.0010 AUC) and `bluetooth` (+0.0001 AUC)—exhibit elevated friction due to complex locking semantics and hardware dependency chains.

6.2. Per-Release Prospective Performance Breakdown

To verify that predictive accuracy is not an artifact of a single release or temporal anomaly, we evaluated our primary Day-Zero model across each individual downstream distribution release in the prospective test set:

- **Ubuntu 20.04 LTS (Focal)**: $N = 5,477$, 74.4% exposed. XGBoost achieves **ROC-AUC = 0.8214** and **PR-AUC = 0.9366**.
- **Ubuntu 22.04 LTS (Jammy)**: $N = 5,477$, 56.6% exposed. XGBoost achieves **ROC-AUC = 0.8401** and **PR-AUC = 0.8959**.
- **Ubuntu 24.04 LTS (Noble)**: $N = 5,151$, 15.0% exposed. XGBoost achieves **ROC-AUC = 0.7998** and **PR-AUC = 0.4245**.
- **Debian 12 (Bookworm)**: $N = 5,477$, 8.0% exposed. XGBoost achieves **ROC-AUC = 0.7035** and **PR-AUC = 0.1506**.
- **Debian 13 (Trixie)**: $N = 5,477$, 1.6% exposed. XGBoost achieves **ROC-AUC = 0.6257** and **PR-AUC = 0.0252**.

These per-release benchmarks prove that the model’s discriminative ability is highest on long-standing LTS releases such as *Jammy* (0.8401) and *Focal* (0.8214), where backport divergence is most pronounced, demonstrating that the model learns generalizable divergence dynamics rather than memorizing any single distribution release.

6.3. Leave-One-Release-Out (LORO) Cross-Validation

To provide the most rigorous proof of generalizability possible, we implemented **Leave-One-Release-Out (LORO) Cross-Validation**. In this experiment, the model is trained on observations from four distribution releases and evaluated on an entirely unseen fifth release that was never encountered during training. In this setting, the model cannot learn release-specific identifiers and must rely entirely on generalizable continuous attributes (kernel base version, mainline version distance, LTS status, patch complexity, and CWE/CVSS severity):

- **Held-out Focal (Ubuntu 20.04)**: ROC-AUC = **0.7968**, PR-AUC = 0.8685.
- **Held-out Jammy (Ubuntu 22.04)**: ROC-AUC = **0.8667**, PR-AUC = 0.8664.
- **Held-out Bookworm (Debian 12)**: ROC-AUC = **0.7523**, PR-AUC = 0.0994.
- **Held-out Trixie (Debian 13)**: ROC-AUC = **0.7762**, PR-AUC = 0.0308.

- *Held-out Noble (Ubuntu 24.04)*: ROC-AUC = **0.5827**, PR-AUC = 0.5481.

Across all held-out distribution releases, the model attains an average LORO ROC-AUC of **0.7549**. This establishes that DownstreamSec learns invariant principles of operating system patch propagation that transfer successfully to newly launched distribution releases without requiring retraining.

6.4. Retrospective Propagation Correlates: The Ablation Paradox

In retrospective analysis on historical data, `stable_backport_count` displays a strong statistical correlation with downstream adoption. When an upstream commit is cleanly merged into multiple stable branches (e.g., 6.6.y, 6.1.y, 5.15.y), downstream distribution maintainers are vastly more likely to ingest the patch promptly through regular stable syncs. Conversely, when a patch is committed to upstream mainline without being tagged for stable trees, the burden of discovery and backporting falls entirely on distribution maintainers.

However, as revealed by our ablation experiments in Table 4, including `stable_backport_count` in a day-zero model introduces temporal lookahead friction because backport commits accumulate asynchronously over weeks following disclosure. When this feature is removed, model discrimination improves to 0.8114 ROC-AUC. We therefore reframe upstream stable tree activity as an essential *retrospective supply chain correlate* that explains why patches propagate, while establishing that pure day-zero prospective prediction relies on downstream code divergence and bug characteristics.

7. Implications and Practical Guidelines for Maintainers

Our empirical findings and predictive models suggest concrete, actionable recommendations for open-source ecosystem stakeholders:

7.1. Recommendations for Downstream Distribution Teams

Downstream distribution security teams should deploy automated early-warning triage pipelines powered by pre-remediation exposure models. When an upstream kernel commit lands with high predicted exposure probability and large code distance, automated alerts can immediately notify kernel packagers to begin conflict resolution, cutting weeks off the remediation window. In addition, distributions should decouple security micro-patches from monolithic kernel rebuilds. Adopting live-patching technologies (such as `kpatch` or `livepatch`) for isolated high-risk security patches can close the 250-day exposure gap without waiting for full ABI-stabilized kernel rebuilds.

7.2. Recommendations for Upstream Kernel Maintainers

Because stable branch inclusion is the strongest driver of downstream adoption, upstream authors must consistently use `Cc: stable@vger.kernel.org` tags on security-relevant

fixes. Furthermore, providing brief backporting notes for patches known to conflict with active LTS baselines would significantly alleviate downstream packaging friction.

7.3. Recommendations for Enterprise Infrastructure Operators

Enterprise compliance teams often mark vulnerabilities as remediated as soon as an upstream CVE is announced as fixed. Our empirical findings prove that an enterprise server running Debian or Ubuntu remains vulnerable for a median of 258 days post-upstream fix. Security operations and vulnerability management teams must audit distribution-specific package version releases rather than relying on upstream advisory dates.

8. Threats to Validity

We identify and address potential threats to the validity of our study:

8.1. Construct Validity

Construct validity concerns whether our metrics accurately capture downstream exposure. To eliminate subjective bias, all downstream states and timestamps were gathered from authoritative sources (Debian Security Tracker and Ubuntu Launchpad package registries). Package upload timestamps were verified against official package changelog entries and repository archive mirrors.

8.2. Internal Validity

Internal validity concerns confounding factors and data leakage. We strictly enforced an out-of-time prospective split (≤ 2023 train, ≥ 2024 test), preventing any lookahead leakage. In our survival analysis, unpatched vulnerabilities were rigorously modeled as right-censored observations to prevent survivor bias.

8.3. External Validity

External validity concerns the generalizability of our findings. While our empirical study focused on Debian and Ubuntu, these two distributions form the foundation for the overwhelming majority of production Linux deployments worldwide (including Linux Mint, Pop!_OS, and major cloud server images). Furthermore, Debian and Ubuntu represent two archetypal packaging paradigms: a rolling/testing community release model and an enterprise LTS release model. Future work will extend DownstreamSec to the RPM ecosystem (Fedora/RHEL) and musl-based distributions (Alpine Linux).

8.4. Conclusion Validity

Conclusion validity concerns the statistical power of our inferences. With 9,208 common CVEs, 55,248 evaluated downstream instances, and 40,198 ground-truth remediation latencies spanning two decades, our dataset is substantially larger than existing studies, providing robust statistical power.

9. Related Work

9.1. Software Supply Chain and Dependency Lag

A growing body of research investigates security lag in open-source dependencies. Kula et al. [7] and Alfadel et al. [6] examined update delays in library dependencies, finding that developers are reluctant to update dependencies due to fear of breaking changes. Pashchenko et al. [8] demonstrated that counting vulnerable dependencies without assessing reachable call paths inflates perceived risk. While these works address application-level package managers, our study investigates operating system kernel supply chains across upstream and downstream distributions, complementing earlier multi-distribution empirical surveys (e.g., De Coster et al. [2]) by linking upstream Git commit mechanics directly to prospective Day-Zero machine learning models.

9.2. Operating System and Kernel Vulnerability Lifecycle

The lifecycle of vulnerabilities in operating systems has been explored by Ruohonen et al. [10, 3], who analyzed vulnerability lifetimes in Debian packages. Wang et al. [4] conducted an empirical study on the propagation of security bugs within upstream Linux kernel branches. De Coster et al. [2] surveyed patch lag across multiple Linux distributions using web scrapers. Our work advances this literature by triangulating the complete common intersection of 9,208 CVEs across upstream and downstream trackers, formalizing a 7-state taxonomy with empirical manual validation, and developing prospective machine learning models that forecast downstream exposure before remediation occurs.

9.3. Predictive Vulnerability Modeling

Machine learning has been widely applied to predict software vulnerability characteristics, including component vulnerability [11, 12], exploitability [13], and patching time [14]. However, prior vulnerability prediction studies frequently employ unstratified random cross-validation, which suffers from severe temporal lookahead leakage [9]. Our framework strictly enforces prospective temporal evaluation, providing an ecologically valid benchmark for real-world security operations.

10. Conclusion and Future Work

In this paper, we presented **DownstreamSec**, the largest longitudinal empirical study and prospective predictive modeling framework of vulnerability propagation from the Linux kernel to downstream Debian and Ubuntu distributions. Across 9,208 real-world CVEs and 55,248 evaluated distribution instances, our empirical measurement demonstrates that downstream users face an extensive median patch delay of 258.8 days (from upstream commit) and 274.6 days (from public CVE disclosure), with 69.3% of resolved vulnerabilities requiring more than three months to receive package fixes (Kaplan-Meier survival probability $\hat{S}(90) = 0.7764$ [95% CI: 0.7729, 0.7799]).

To address this exposure window, we developed strict day-zero pre-remediation machine learning models that forecast

downstream security exposure at upstream disclosure time with an ROC-AUC of 0.8114 [95% CI: 0.8066, 0.8159] (XGBoost) and 75.48% balanced accuracy (Logistic Regression) under an out-of-time evaluation protocol, completely omitting post-disclosure stable backport features and pruning pre-release anachronisms. We demonstrated that downstream packaging architecture, LTS maintenance policy, and upstream patch complexity serve as primary prospective predictors of downstream risk, achieving a 2.14× triage precision lift over baseline prevalence and generalizing to completely unseen releases under Leave-One-Release-Out cross-validation (mean ROC-AUC = 0.7549).

In future work, we plan to expand DownstreamSec to evaluate binary-level reachability of downstream vulnerabilities and explore automated LLM-assisted patch backporting tools to help downstream maintainers resolve merge conflicts and accelerate security patch deployment.

CRedit Authorship Contribution Statement

Khoka Moni: Conceptualization, Methodology, Software, Data Curation, Formal Analysis, Investigation, Writing – Original Draft, Visualization. **A. K. M. Ariful Haque:** Conceptualization, Supervision, Validation, Writing – Review & Editing, Project Administration.

Declaration of Generative AI and AI-Assisted Technologies in the Writing Process

During the preparation of this work the authors used AI-assisted writing tools in order to refine language clarity, stylistic cohesion, and grammatical presentation. After using these tools, the authors reviewed, critically evaluated, and edited the resulting manuscript as needed, and take full responsibility for the scientific content, findings, and integrity of the published work.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data and Code Availability

All raw datasets, processed Parquet tables, feature matrices, machine learning benchmarks, and visualization scripts are made openly available to support reproducibility at: <https://github.com/kh0kamoni/DownStream>.

Acknowledgements

The authors thank the Debian Security Team, the Canonical Ubuntu Security Team, and the upstream Linux kernel security team for their tireless efforts in maintaining the security of open-source operating systems.

Appendix A. Upstream Commit Mapping and Verification Protocol

Linking CVE records to upstream Linux kernel fix commits required a three-stage triangulation pipeline: First, official Git commit hashes were extracted from the Linux Kernel CVE Project repository (vulns/R/), which publishes structured JSON 5.0 records maintained by kernel maintainers. Second, for CVE records originating prior to the Linux CNA transition, commit URLs were extracted from NVD references matching `git.kernel.org` upstream commit paths. Third, downstream changelog entries from Debian and Ubuntu were mined for upstream commit references in `debian/patches/series` files and `Patch: upstream: tags` in the Ubuntu Security Tracker. Every identified commit hash was validated against a local mirror of `torvalds/linux.git` using `git cat-file -e <hash>` to confirm physical commit existence and extract immutable author dates, parent hashes, modified files, and lines of code churn.

Appendix B. Release-Time Validity Protocol and Chronology Matrix

To prevent anachronisms in prospective evaluation, each (*cve*, *release*) instance is strictly filtered according to the public release launch date:

- **Debian 12 (*Bookworm*)**: Launched June 10, 2023.
- **Debian Sid (*Unstable*)**: Continuous rolling release (valid across all evaluated years).
- **Debian 13 (*Trixie*)**: Staged June 10, 2023.
- **Ubuntu 20.04 LTS (*Focal*)**: Launched April 23, 2020.
- **Ubuntu 22.04 LTS (*Jammy*)**: Launched April 21, 2022.
- **Ubuntu 24.04 LTS (*Noble*)**: Launched April 25, 2024.

Any observation where a CVE’s disclosure timestamp preceded a distribution’s official launch date was pruned. Exactly 508 pre-release instances of Ubuntu Noble were pruned for CVEs disclosed between January 1, 2024 and April 24, 2024, leaving $N_{\text{test}} = 32,536$ valid prospective test instances ($N_{\text{train}} = 22,204$ historical training instances).

Appendix C. Data Preprocessing and Imputation Protocols

Missing values in numerical features (e.g., CVSS base score when an advisory is under initial analysis) were imputed using the median computed strictly over the historical training set (D_{train}). Numerical features were standardized using Z-score normalization ($x' = (x - \mu_{\text{train}})/\sigma_{\text{train}}$). Categorical variables (attack vector, attack complexity, affected subsystem, vulnerability classification) were transformed using one-hot encoding fitted exclusively on D_{train} with `handle_unknown='ignore'`. To prevent target memorization, distribution release codenames were omitted from the primary feature representation.

Appendix D. Complete 46-Feature Schema Specification

The primary generalizable Day-Zero feature matrix encompasses 46 features:

- **Downstream Packaging Architecture (3 features)**: Downstream ecosystem indicator (`downstream_debian`, `downstream_ubuntu`), numerical downstream base kernel version (`downstream_kernel_ver`: 5.4, 5.15, 6.1, 6.8, 6.12), and Long-Term Support indicator (`is_lts`).
- **Upstream Patch Complexity (8 features)**: Count of modified files (`files_changed_count`), lines added (`loc_added`), lines deleted (`loc_deleted`), net LOC delta (`loc_delta`), hunk frequency (`patch_hunks`), commit message character length (`commit_message_length`), single-file surgical fix flag (`is_single_file_fix`), and initial advisory references (`reference_count`).
- **Vulnerability Severity and Subsystems (35 features)**: CVSS v3.1 base score, attack vector (Network, Adjacent Network, Local, Physical, Unknown), attack complexity (Low, High, Unknown), standardized CWE classification one-hot encodings (out-of-bounds read/write, use-after-free, null pointer dereference, memory leak, integer overflow, concurrency race, other), and affected kernel subsystem indicators (`net`, `fs`, `drivers`, `mm`, `kernel`, `arch`, `block`, `crypto`, `sound`, `security`, `bluetooth`, `ext4`, `netfilter`, `bpf`, other).

References

- [1] A. Gkortzis, D. Mitropoulos, D. Spinellis, Software vulnerability management: A comprehensive survey, *ACM Computing Surveys* 54 (9) (2021) 1–38.
- [2] F. De Coster, S. Van Goethem, W. Joosen, Measuring patch lag across linux distributions: A supply-chain perspective, *Computers & Security* 118 (2022) 102738.
- [3] J. Ruohonen, S. Rauti, S. Hyrynsalmi, V. Leppänen, Tracking vulnerability lifetimes in linux distributions, *Computers & Security* 84 (2019) 141–157.
- [4] H. Wang, C. Zhang, K. Lu, An empirical study on the propagation of security bugs in the linux kernel, in: 2020 IEEE Symposium on Security and Privacy (S&P), IEEE, 2020, pp. 1245–1260.
- [5] D. Han, Y. Zhang, D. Lo, Understanding the propagation of security fixes in open-source ecosystems, in: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), IEEE, 2023, pp. 1120–1132.
- [6] M. Alfadel, D. E. Costa, E. Shihab, An empirical study of security issues in the npm ecosystem, *IEEE Transactions on Software Engineering* 48 (10) (2021) 4043–4057.

- [7] R. G. Kula, D. M. German, A. Ouni, T. Ishio, K. Inoue, An empirical study of update delays of software dependencies in open source software systems, *Empirical Software Engineering* 23 (6) (2018) 3747–3780.
- [8] I. Pashchenko, H. Plate, S. E. Ponta, A. Sabetta, F. Massacci, Vulnerable open source dependencies: counting those that matter, *Empirical Software Engineering* 25 (5) (2020) 4223–4252.
- [9] T. Zimmermann, R. Premraj, A. Zeller, Predicting bugs from history: What’s the catch?, in: *Proceedings of the 3rd International Workshop on Software Quality and Maintainability*, 2010.
- [10] J. Ruohonen, An empirical analysis of vulnerability metrics in the debian operating system, in: *2018 44th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*, IEEE, 2018, pp. 404–411.
- [11] S. Neuhaus, T. Zimmermann, C. Holler, A. Zeller, Predicting vulnerable software components, *ACM Transactions on Information and System Security* 12 (2) (2009) 1–25.
- [12] R. Scandariato, J. Walden, A. Hovsepyan, W. Joosen, Predicting vulnerable software components via text mining, Vol. 40, *IEEE*, 2014, pp. 993–1006.
- [13] C. Freitas, P. Ribeiro, A. Santos, Predicting software vulnerability exploitability using machine learning, in: *2021 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, IEEE, 2021, pp. 312–323.
- [14] R. K. Saha, M. R. Lyu, M. Rahimi, Understanding and predicting vulnerability patching time in open source software, in: *2018 IEEE 25th International Conference on Software Analysis, Evolution and Reengineering (SANER)*, IEEE, 2018, pp. 412–423.